CAPU

Agenda inteligente con gamificación integrada a Google Classroom

EVIDENCIAS PRÁCTICAS Y CAPTURAS DE PANTALLA

Etapa II — Planificación del proyecto · Archivo 2

Tecnológico Nacional de México · Instituto Tecnológico de Tijuana
Materia: Gestión de Proyectos de Software
Versión del software: 0.2.0 (etiqueta v0.2.0) · Fecha de corte: 14 de octubre de 2026

Integrantes del equipo

Ballesteros Cruz Aldo Juventino — #23211920
Bojorquez Valdez Victor Manuel — #23211926
Duran Ponce Luis Adao Leonel — #23211951
Grande Ortega Maximiliano Alberto — #23211974
Lopez Molgado Jorge Luis — #23212002

Contenido

1. Repositorio del proyecto
2. Código fuente desarrollado
3. Base de datos y diagramas técnicos
4. Interfaces y capturas de pantalla de la ejecución
5. Pruebas automatizadas
6. Trazabilidad de requerimientos
7. Problemas detectados y correcciones
8. Cómo reproducir las evidencias

Este documento reúne las evidencias prácticas de la Etapa II: el repositorio, el código fuente, la base de datos, los diagramas, las interfaces y las pruebas. Todo corresponde al proyecto CAPU del Plan de Calidad (Etapa I) y respalda el avance reportado en el Plan del Proyecto.

1. Repositorio del proyecto

1.1 Liga de acceso

Repositorio (GitHub): [PEGAR AQUÍ LA LIGA DEL REPOSITORIO]

El repositorio contiene el código fuente, la base de datos, las pruebas, la documentación y el historial de cambios. Se entrega además la copia completa del repositorio (con su historial de Git) en el archivo CAPU_repositorio.zip.

1.2 Estructura del repositorio

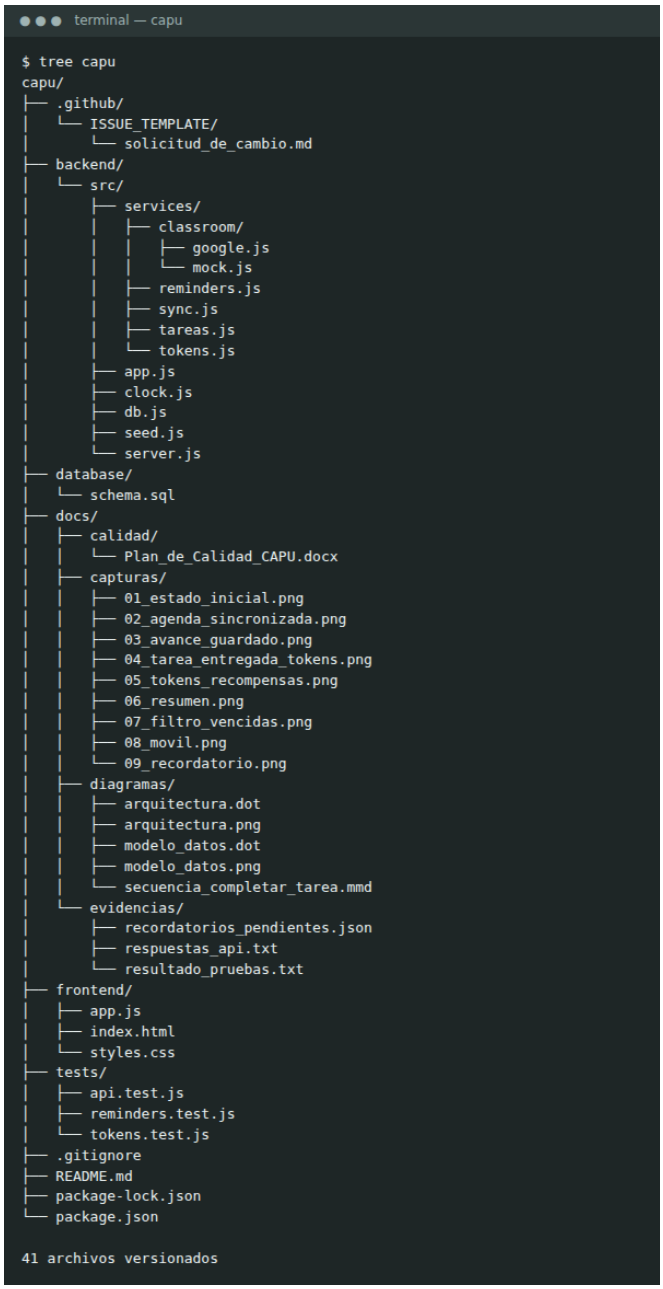


Figura 1. Estructura del repositorio (captura de la terminal).

Organización: backend/ (API y servicios), frontend/ (interfaz), database/ (esquema), tests/ (pruebas), docs/ (calidad, diagramas, capturas y evidencias) y .github/ (plantilla de solicitud de cambio).

1.3 Historial de cambios y versiones

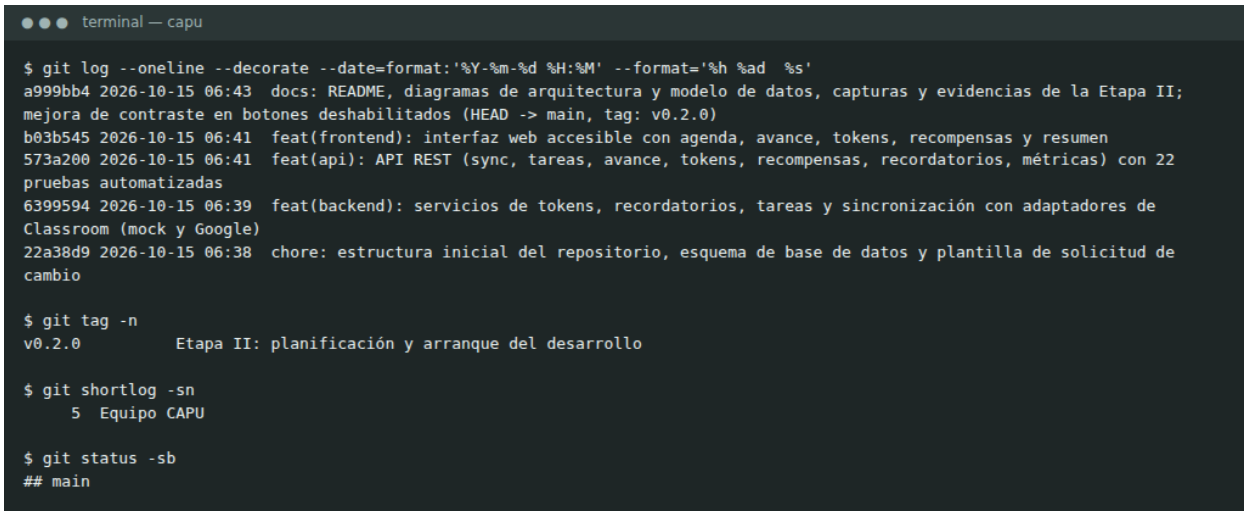


Figura 2. Historial de commits, versión etiquetada (v0.2.0) y estado del repositorio.

Commit	Fecha y hora (UTC)	Autor	Mensaje
22a38d9	2026-10-15 06:38	Equipo CAPU	chore: estructura inicial del repositorio, esquema de base de datos y plantilla de solicitud de cambio
6399594	2026-10-15 06:39	Equipo CAPU	feat(backend): servicios de tokens, recordatorios, tareas y sincronización con adaptadores de Classroom (mock y Google)
573a200	2026-10-15 06:41	Equipo CAPU	feat(api): API REST (sync, tareas, avance, tokens, recompensas, recordatorios, métricas) con 22 pruebas automatizadas
b03b545	2026-10-15 06:41	Equipo CAPU	feat(frontend): interfaz web accesible con agenda, avance, tokens, recompensas y resumen
a999bb4	2026-10-15 06:43	Equipo CAPU	docs: README, diagramas de arquitectura y modelo de datos, capturas y evidencias de la Etapa II; mejora de contraste en botones deshabilitados

Versionado: se usa versionado semántico. La versión 0.2.0 (etiqueta v0.2.0) corresponde al cierre de esta etapa; cada commit sigue la convención tipo(ámbito): descripción.

1.4 README actualizado

El archivo README.md del repositorio incluye: nombre del proyecto, integrantes y roles, tecnologías, instrucciones de instalación y ejecución, credenciales de prueba (no se requieren: modo demostración), variables de entorno, ejecución de pruebas, estructura, resumen de la API, estado de los requerimientos y flujo de trabajo.

1.5 Métricas del repositorio

Concepto	Valor
Commits	5
Archivos versionados	41
Líneas de código (módulos de la sección 2)	870
Pruebas automatizadas	22 (todas satisfactorias)
Rutas de la API	12
Tablas en la base de datos	7

Nota sobre autoría: los commits iniciales se registraron con la identidad común del equipo. A partir de la publicación del repositorio, cada integrante trabaja en su rama y firma sus commits con su cuenta personal, de modo que el historial sirva como evidencia individual de participación.

2. Código fuente desarrollado

El código fuente completo está en el repositorio. A la fecha de corte suma 870 líneas distribuidas en los siguientes módulos:

Archivo	Función	Requerimientos	Líneas
backend/src/app.js	API REST (12 rutas), manejo de errores	RF-03 a RF-12	72
backend/src/db.js	Conexión SQLite y transacciones anidadas	RNF-08	34
backend/src/services/tokens.js	Reglas de tokens, racha y canje	RF-07, RF-08, RF-09	82
backend/src/services/reminders.js	Cálculo y programación de recordatorios	RF-06	36
backend/src/services/sync.js	Sincronización de cursos y tareas	RF-03	45
backend/src/services/tareas.js	Agenda, avance y métricas	RF-04, RF-05, RF-12	42
backend/src/services/classroom/mock.js	Adaptador de demostración	RNF-06	34
backend/src/services/classroom/google.js	Adaptador de Google Classroom (lectura)	RF-02, RF-03	36
database/schema.sql	Esquema relacional de 7 tablas	Modelo de datos	71
frontend/index.html, styles.css, app.js	Interfaz: agenda, tokens, resumen	RF-04 a RF-10, RNF-03, RNF-04	244
tests/*.test.js	22 pruebas automatizadas	Calidad	174

2.1 Fragmento: regla de tokens (RF-07)

Función pura que calcula los tokens por una entrega. Está cubierta por 9 pruebas de reglas de negocio.

```
function calcularTokens({ fechaLimite, fechaEntrega, racha = 0 }) {
  const limite = new Date(fechaLimite).getTime();
  const entrega = new Date(fechaEntrega).getTime();
  if (entrega > limite) return { cantidad: 0, aTiempo: false, motivo: 'Entrega tardía: sin
tokens' };
  const partes = [`base ${BASE}`];
  let cantidad = BASE;
  if (limite - entrega >= HORAS_ANTICIPADA * 3600e3) { cantidad += BONO_ANTICIPADA;
partes.push(`anticipada +${BONO_ANTICIPADA}`); }
  const bonoRacha = Math.min(Math.max(racha, 0), BONO_RACHA_MAX);
  if (bonoRacha > 0) { cantidad += bonoRacha; partes.push(`racha +${bonoRacha}`); }
  return { cantidad, aTiempo: true, motivo: `Entrega a tiempo (${partes.join(', ')})` };
}
```

Fragmento de backend/src/services/tokens.js.

2.2 Fragmento: sincronización con Classroom (RF-03)

La API elige el adaptador: con un token de Google usa el adaptador real; sin él usa datos de demostración.

```
  app.post('/api/sync', wrap(async (req) => {
```

```
    const token = req.get('x-google-token');
    const adaptador = token ? googleAdapter(token) : mockAdapter(now());
    return { adaptador: adaptador.nombre, ...(await sincronizar(db, adaptador, now())) };
  }));
```

Fragmento de backend/src/app.js.

2.3 Fragmento: adaptador de Google Classroom (RF-02, RF-03)

```
    async obtenerTareas(idCurso) {
      const [w, s] = await Promise.all([
        get(`/courses/${idCurso}/courseWork`),
        get(`/courses/${idCurso}/courseWork/-/studentSubmissions?userId=me`),
      ]);
      const entregas = new Map((s.studentSubmissions || []).map((e) => [e.courseWorkId, e]));
      return (w.courseWork || []).filter((t) => t.dueDate).map((t) => {
        const e = entregas.get(t.id);
        const entregada = !!e && ['TURNED_IN', 'RETURNED'].includes(e.state);
        return { id: t.id, titulo: t.title, descripcion: t.description || '', fechaLimite:
aIso(t.dueDate, t.dueTime),
          entregada, fechaEntrega: entregada ? e.updateTime : null, enlace:
t.alternateLink };
      });
    },
  };
```

Fragmento de backend/src/services/classroom/google.js. Probado con respuestas simuladas; falta la prueba con credenciales reales.

3. Base de datos y diagramas técnicos

3.1 Esquema de base de datos

El esquema (database/schema.sql) define siete tablas con llaves foráneas, restricciones de integridad (por ejemplo, saldo de tokens no negativo y avance entre 0 y 100) e índices para las consultas de agenda y recordatorios.

```
CREATE TABLE IF NOT EXISTS curso (
  id_curso      INTEGER PRIMARY KEY AUTOINCREMENT,
  id_usuario    INTEGER NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE,
  classroom_id  TEXT NOT NULL,
  nombre        TEXT NOT NULL,
  UNIQUE (id_usuario, classroom_id)
);

CREATE TABLE IF NOT EXISTS tarea (
  id_tarea      INTEGER PRIMARY KEY AUTOINCREMENT,
  id_curso      INTEGER NOT NULL REFERENCES curso(id_curso) ON DELETE CASCADE,
  classroom_id  TEXT NOT NULL,
  titulo        TEXT NOT NULL,
  descripcion   TEXT,
  fecha_limite  TEXT NOT NULL,      -- ISO 8601 (UTC)
  estado        TEXT NOT NULL DEFAULT 'pendiente'
                CHECK (estado IN ('pendiente','en_progreso','entregada')),
  avance_pct    INTEGER NOT NULL DEFAULT 0 CHECK (avance_pct BETWEEN 0 AND 100),
  fecha_entrega TEXT,
  enlace        TEXT,               -- enlace directo a la tarea en Google Classroom (RF-10)
```

```
  UNIQUE (id_curso, classroom_id)
);
```

Fragmento de database/schema.sql: tablas curso y tarea.

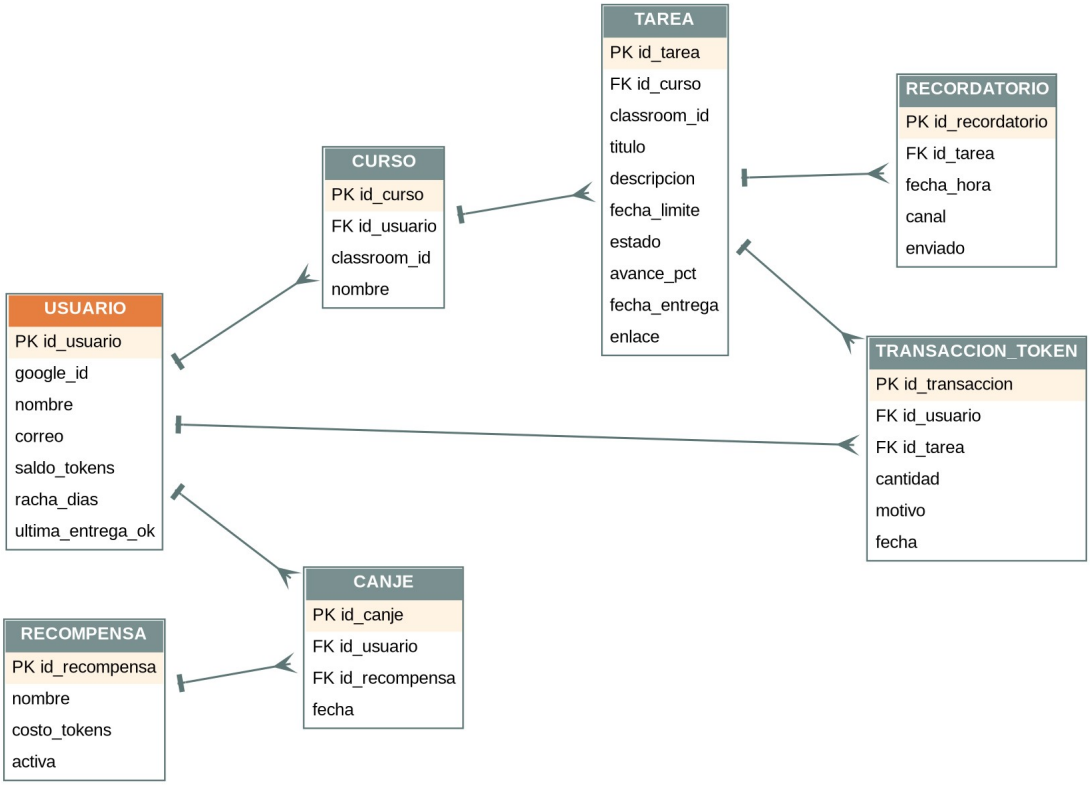


Figura 11. Modelo entidad-relación implementado.

3.2 Arquitectura

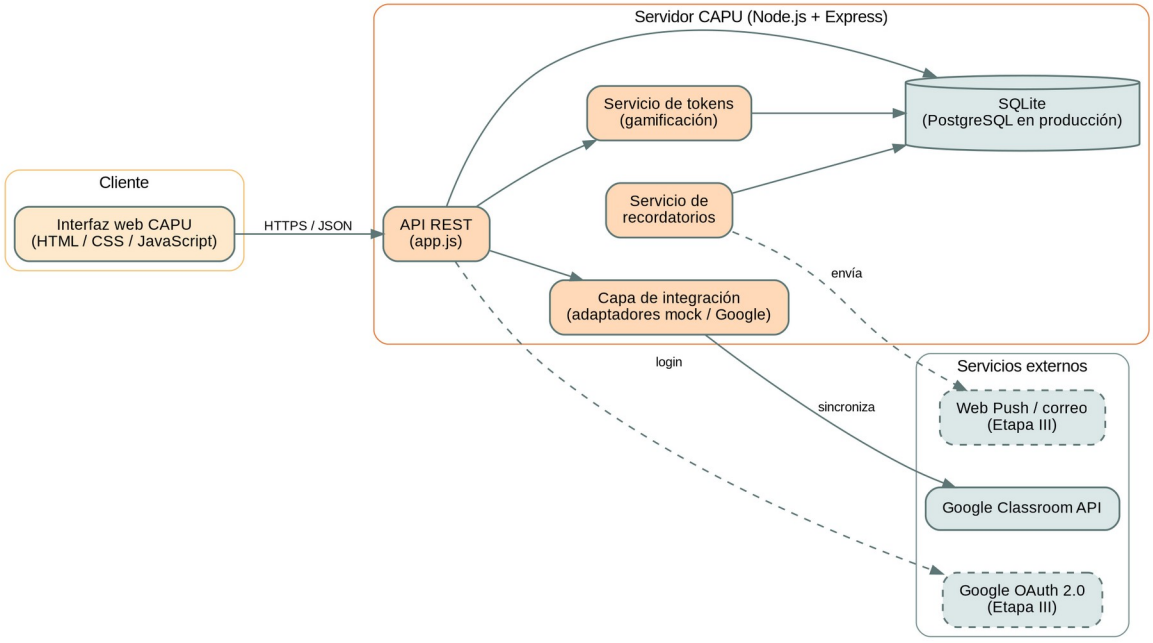


Figura 12. Arquitectura del sistema. Líneas punteadas: componentes de la Etapa III (OAuth y notificaciones).

4. Interfaces y capturas de pantalla de la ejecución

Las capturas se tomaron del sistema en ejecución (versión 0.2.0), operando con datos de demostración y la fecha del sistema fijada en el 14 de octubre de 2026. Cada captura corresponde a una interacción real con la interfaz.

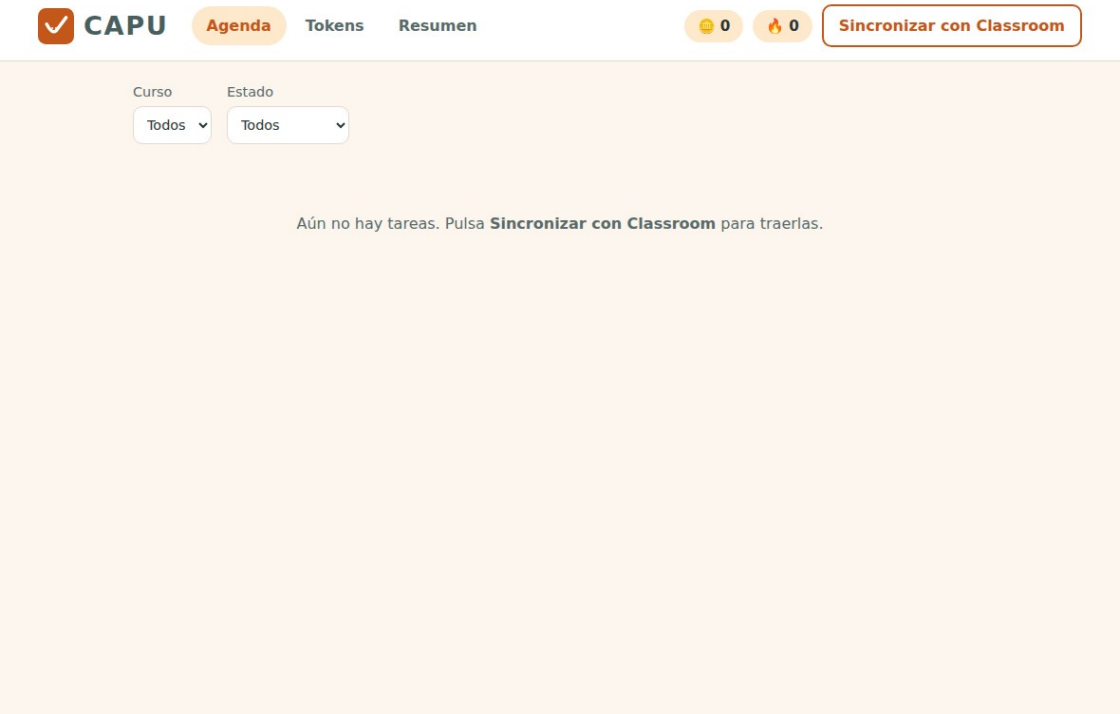


Figura 3. Estado inicial: sin tareas. La aplicación invita a sincronizar con Classroom.

Muestra la pantalla vacía antes de sincronizar (RF-04).

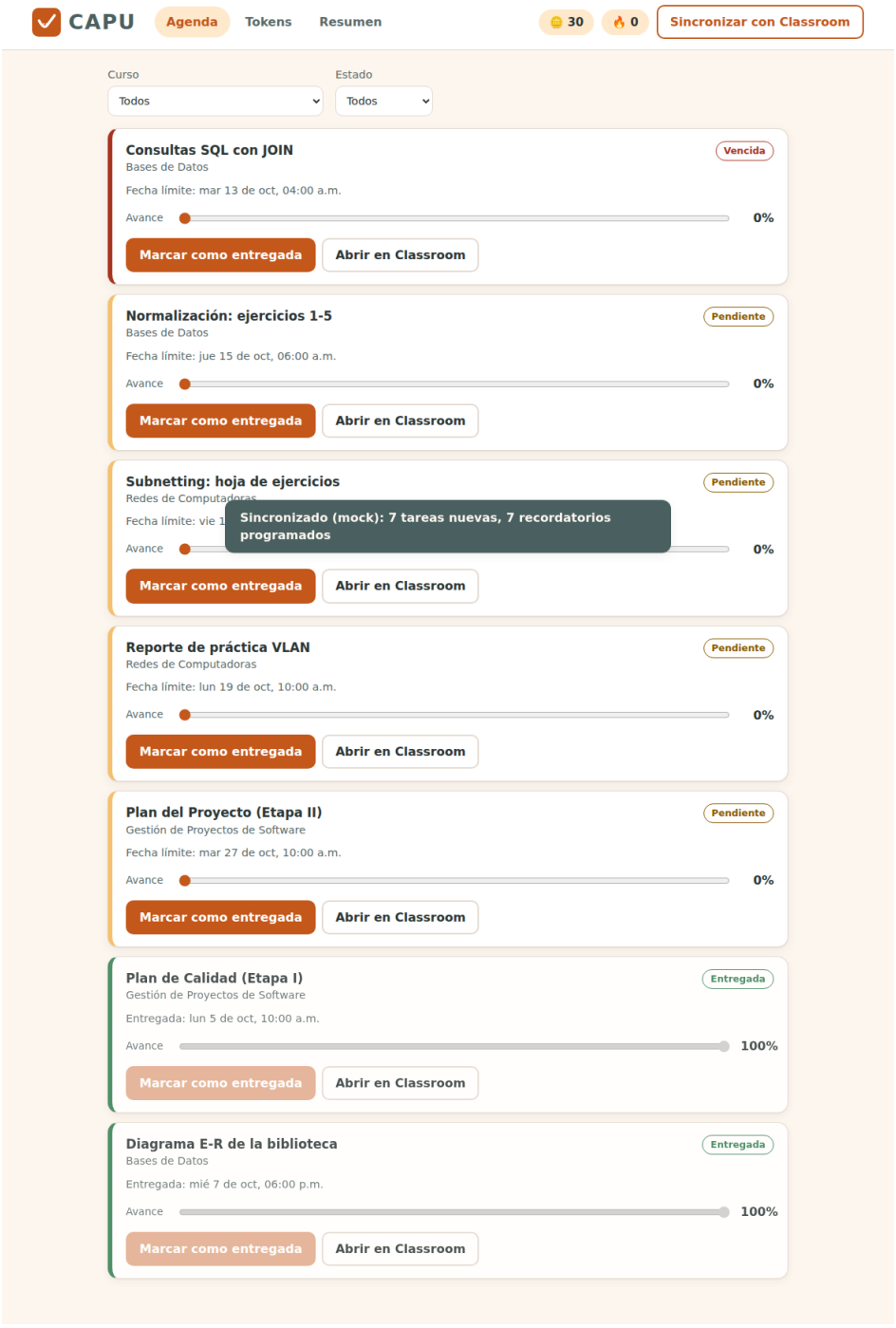


Figura 4. Agenda tras la sincronización (modo de demostración).

Se importaron 3 cursos y 7 tareas. Cada tarjeta muestra curso, fecha límite, estado (Vencida, Pendiente, Entregada), barra de avance y acceso a Classroom. Las dos tareas ya entregadas en Classroom se registraron con tokens (RF-03, RF-04, RF-10).

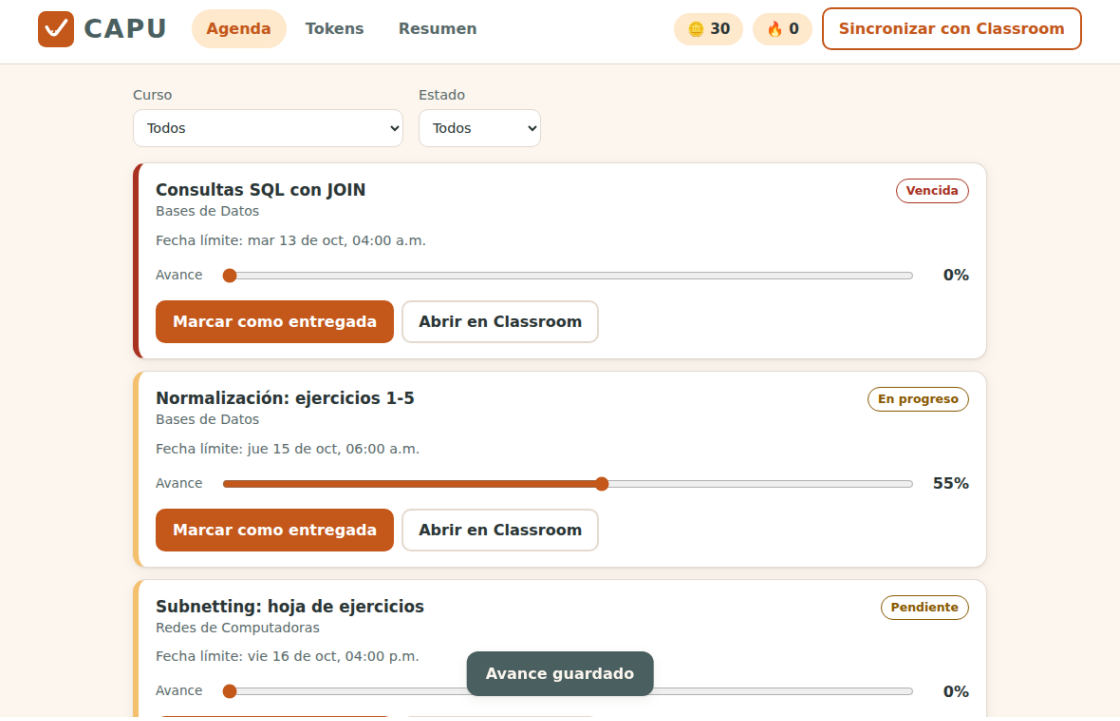


Figura 5. Registro del avance de una tarea (60 %).

Se movió el control de avance con el teclado; el estado cambió a “En progreso” y el avance se guardó en la base de datos (RF-05).

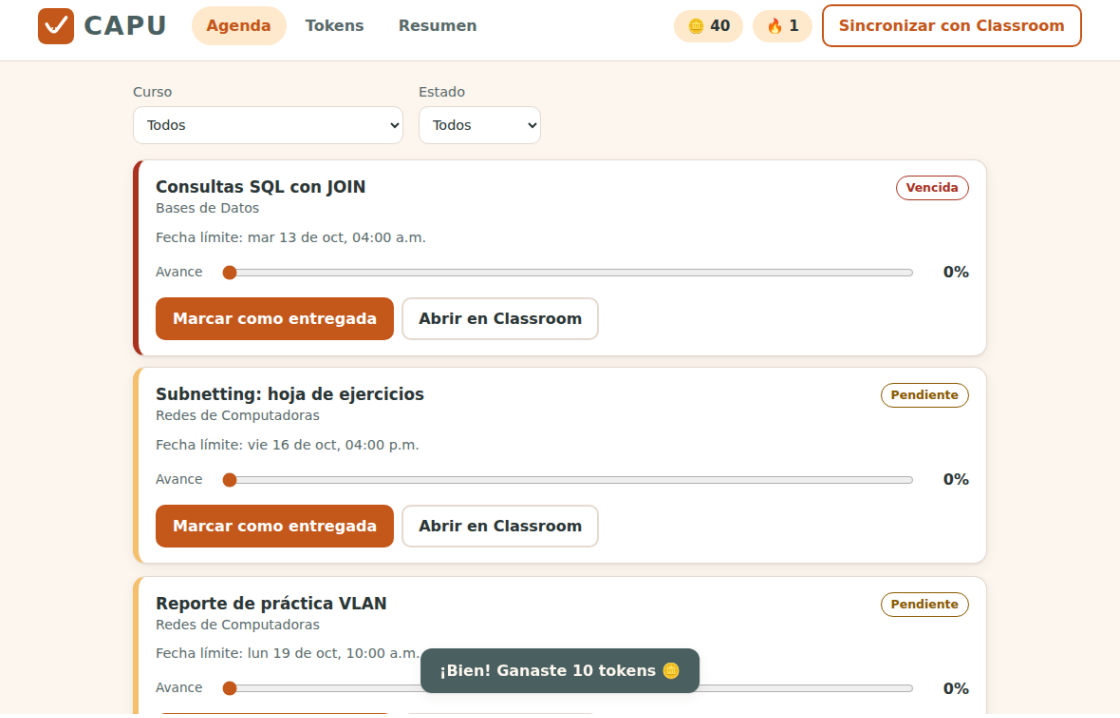


Figura 6. Tarea marcada como entregada y tokens otorgados.

El aviso confirma los tokens ganados, y el contador de la cabecera (saldo y racha) se actualiza (RF-07).

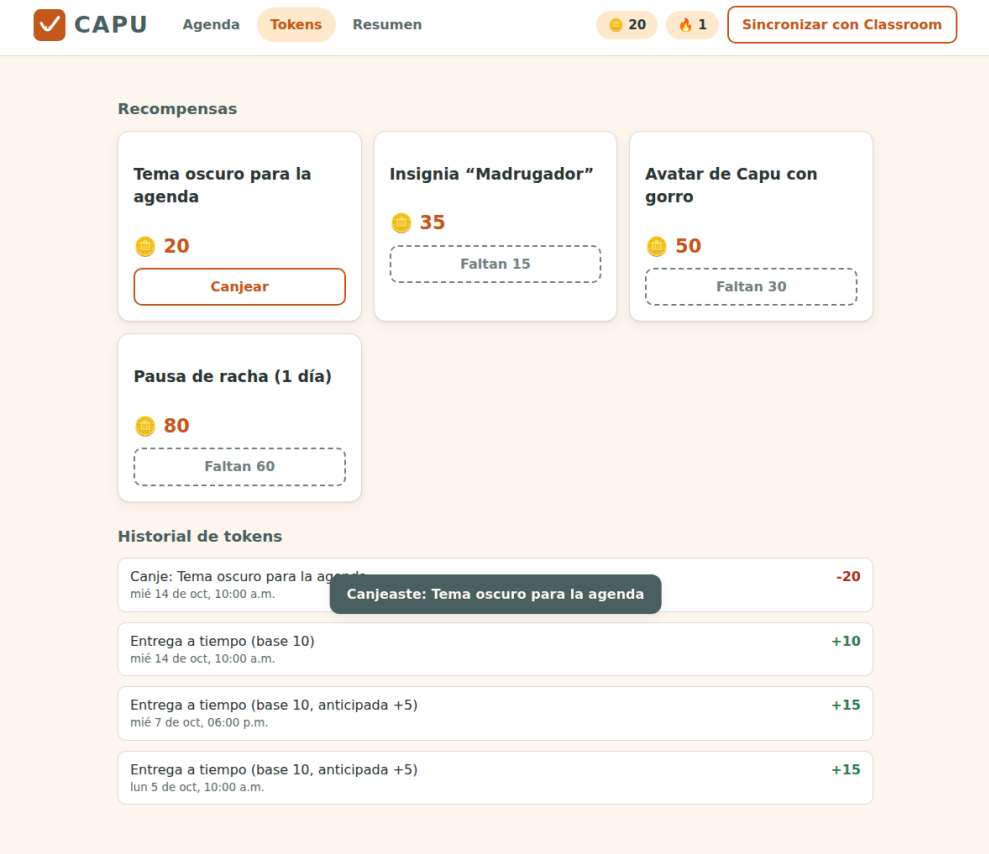


Figura 7. Vista de tokens: recompensas e historial.

Se canjeó “Tema oscuro” por 20 tokens. El historial muestra tres ganancias y un canje; las recompensas que no alcanzan indican cuántos tokens faltan (RF-08, RF-09).



Figura 8. Resumen de desempeño.

Indicadores: porcentaje de tareas entregadas a tiempo, pendientes, vencidas, tokens y racha (RF-12; indicadores de seguimiento del Plan de Calidad).

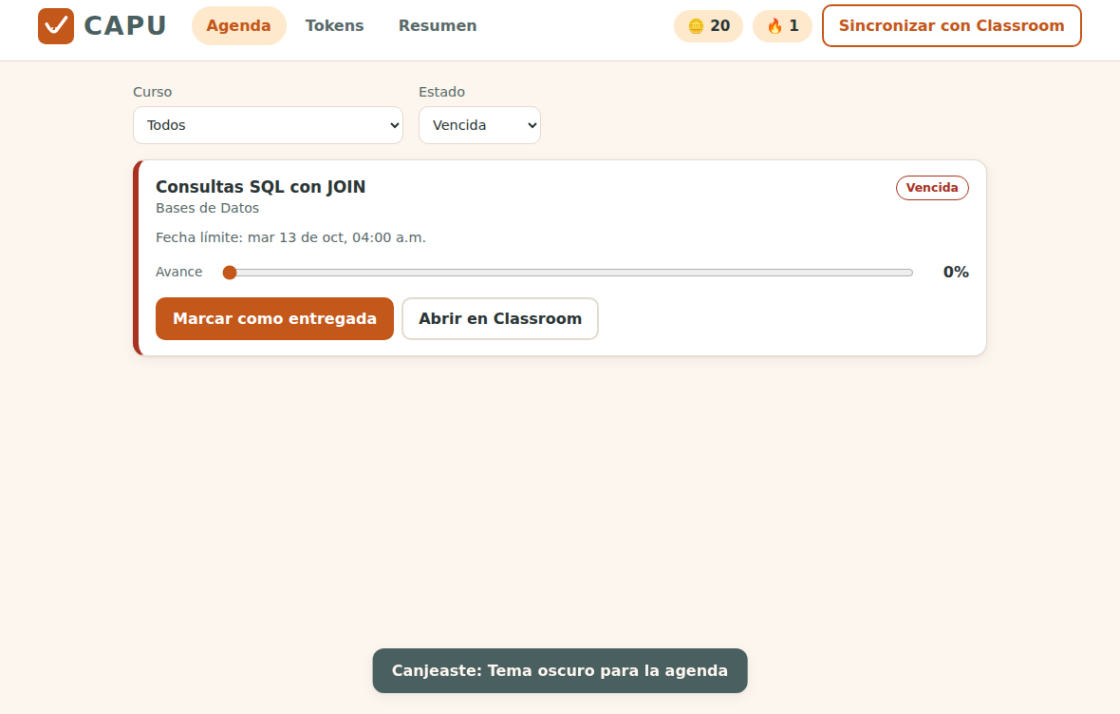


Figura 9. Filtro por estado “Vencida”.

La tarea “Consultas SQL con JOIN” supera su fecha límite sin entrega y se muestra como vencida (RF-04).



Figura 10. Recordatorio vencido en la agenda.

Con el reloj en el 16 de octubre, el recordatorio programado 24 h antes de “Subnetting” aparece como aviso (RF-06; el envío por Web Push o correo se implementa en la Etapa III).

4.1 Versión adaptable y accesibilidad

La interfaz se adapta a pantallas pequeñas y respeta el modo oscuro del sistema. Incluye etiquetas en los controles, foco visible, enlace para saltar al contenido, regiones de aviso para lectores de pantalla y botones con contraste suficiente (RNF-03, RNF-04). La auditoría con herramienta automática queda programada en la actividad B11.



Figura 13. Vista en pantalla de teléfono (390 px de ancho).

5. Pruebas automatizadas

Se ejecutaron con npm test (ejecutor nativo de Node.js). Resultado: 22 pruebas, 22 satisfactorias, 0 fallidas. Cubren las reglas de tokens, los recordatorios, la API completa y el adaptador de Google con respuestas simuladas.

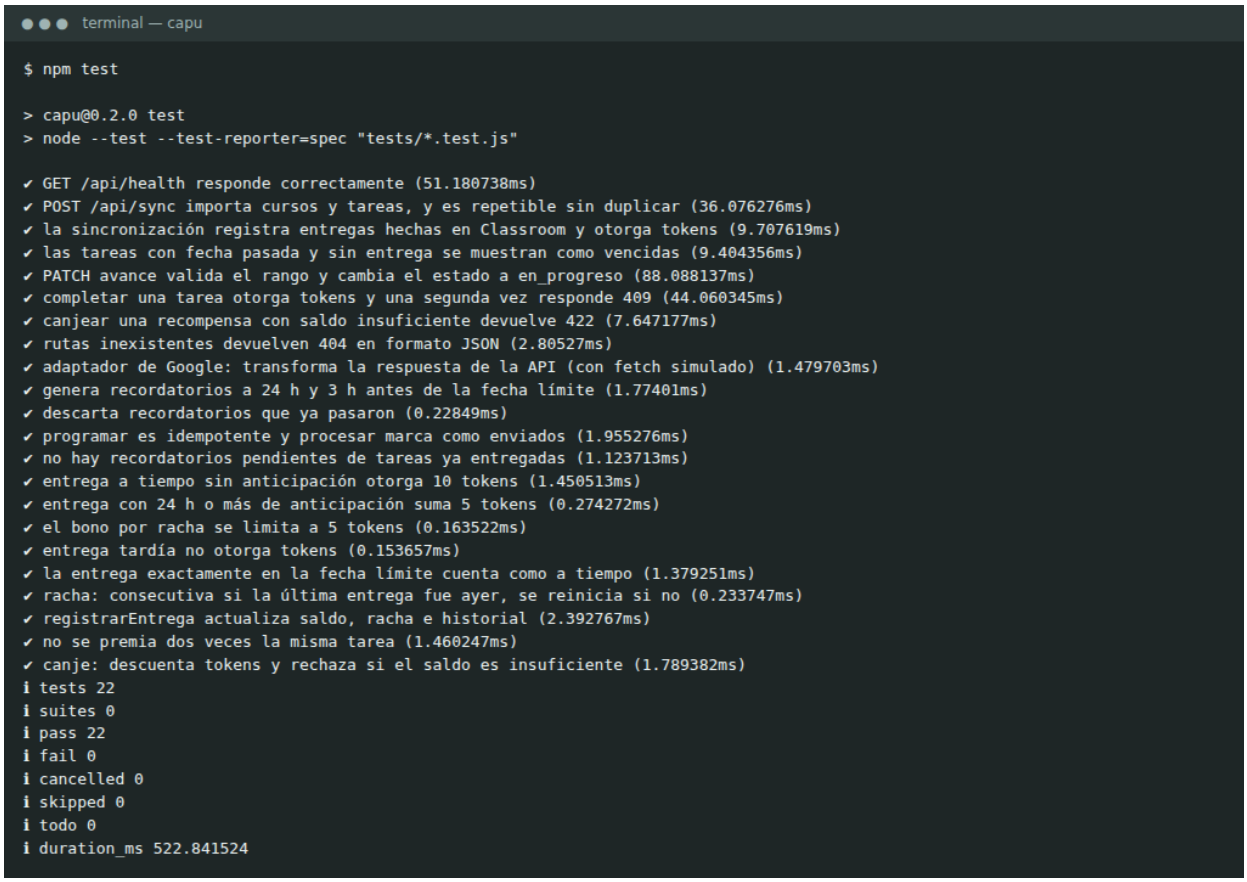


Figura 14. Resultado de la ejecución de las pruebas.

Archivo de pruebas	Pruebas	Qué verifican
tests/tokens.test.js	9	Tokens por entrega a tiempo, anticipada, racha, tardía; no premiar dos veces; canje y saldo insuficiente.
tests/reminders.test.js	4	Cálculo a 24 h y 3 h, descarte de pasados, idempotencia y estado de envío.
tests/api.test.js	9	Salud, sincronización repetible, vencidas, avance, completar, canje, rutas inexistentes y adaptador de Google.

Respuestas de la API en ejecución

```
$ curl /api/health
{"ok":true,"version":"0.2.0","ahora":"2026-10-16T00:00:00.000Z"}

$ curl /api/metricas
{"total":7,"entregadas":3,"a_tiempo":3,"porcentaje_a_tiempo":100,"pendientes":3,"vencidas":1,
"saldo_tokens":2...

$ curl /api/recompensas
[{"id_recompensa":1,"nombre":"Tema oscuro para la agenda","costo_tokens":20,"activa":1},
{"id_recompensa":2,"n...

$ curl -X POST /api/recordatorios/procesar
{"enviados":[{"id_recordatorio":4,"fecha_hora":"2026-10-
15T23:00:00.000Z","canal":"push","id_tarea":6,"titulo...
```

Extracto de docs/evidencias/respuestas_api.txt.

6. Trazabilidad de requerimientos

ID	Requerimiento	Estado	Evidencia
RF-03	Sincronización de tareas	Implementado	sync.js; Figura 4; prueba “POST /api/sync…”
RF-04	Agenda con filtros	Implementado	Figuras 4 y 9; prueba de vencidas
RF-05	Seguimiento de avance	Parcial	Figura 5; prueba de PATCH (subtareas pendientes)
RF-06	Recordatorios	Parcial	reminders.js; Figura 10; 4 pruebas (envío real pendiente)
RF-07	Tokens por cumplimiento	Implementado	tokens.js; Figura 6; 9 pruebas
RF-08	Historial y saldo	Implementado	Figura 7
RF-09	Canje de recompensas	Parcial	Figura 7; prueba de canje (administración del catálogo pendiente)
RF-10	Acceso a la entrega	Implementado	Botón “Abrir en Classroom”, Figura 4
RF-12	Reportes de desempeño	Implementado	Figura 8
RF-01, 02, 11	Inicio de sesión, vinculación, preferencias	Pendiente	Programados en la Etapa III (C1, C3)

7. Problemas detectados y correcciones

Las pruebas y la revisión de la interfaz permitieron detectar y corregir tres problemas antes del cierre de la etapa:

#	Problema	Causa y corrección	Verificación
1	La sincronización fallaba con el error “cannot start a transaction within a transaction”.	Una transacción llamaba a otra. Se reescribió transaction() en db.js con SAVEPOINT para admitir anidamiento.	5 pruebas de API que fallaban pasaron (commit 573a200).
2	La hora de entrega de Google se interpretaba mal (18:59 en lugar de 18:00).	La API omite los campos en cero; se corrigió el valor por defecto de los minutos en google.js.	Prueba del adaptador de Google.
3	Los botones deshabilitados (“Faltan N tokens”) tenían bajo contraste.	Se aumentó la opacidad y se añadió borde punteado y color de texto suave.	Figura 7 (captura posterior a la corrección).

8. Cómo reproducir las evidencias

```
git clone <URL-del-repositorio> capu
cd capu
npm install
npm test        # 22 pruebas
npm start       # abre http://localhost:3000 y pulsa "Sincronizar con Classroom"

# Para repetir las capturas con la fecha usada en este documento:
CAPU_FIXED_NOW=2026-10-14T17:00:00Z npm start
```

Requisito: Node.js 22.5 o superior.